

Prova Técnica: Bolsista de Pesquisa e Desenvolvimento (Python/PyQt5)

GIECAR/UFF

Objetivo: avaliar domínio prático em Python, Orientação a Objetos, Estruturas de Dados, PyQt5, processamento numérico e persistência de dados científicos, com foco em performance (uso de memória), responsividade de UI, qualidade de código e testes. Também é objetivo avaliar a capacidade do candidato de entender um problema de domínio da geofísica e traduzi-lo para uma solução prática de software, mesmo sem conhecimento prévio da área.

1) História do Cliente

Uma empresa de óleo e gás financiou um grupo de pesquisa em geofísica computacional para modernizar o pipeline de pré-processamento sísmico. Historicamente, o filtro passa-baixa aplicado aos levantamentos sísmicos era feito por scripts avulsos, sem interface e sem feedback de progresso: o geofísico disparava o script e só descobria se havia funcionado horas depois. Em um projeto crítico, um processamento travou a estação de trabalho durante a madrugada e ninguém percebeu até a manhã seguinte, atrasando a entrega de um prospect em uma semana. Além disso, o resultado não ficava versionado nem era reaproveitável para outras etapas do fluxo. A diretoria pediu uma aplicação desktop robusta, com interface responsiva, progresso visível, possibilidade de cancelamento e persistência estruturada do resultado.

2) Arquitetura de Referência

Arquitetura mínima sugerida (itens opcionais podem ser implementados como bônus):

```
[Usuário] → [Aplicação Desktop PyQt5] → [Worker em background (QThread / QRunnable)]  
  
↓  
  
[Leitura streaming do .sgy via segyio]  
  
↓  
  
[Persistência incremental: HDF5/Zarr para traços + SQLite/SQLAlchemy para metadados]  
  
(Opcional) [Pool de multiprocessing] para paralelizar a filtragem entre traços  
  
(Opcional) [Cache de espectro de amplitude] antes/depois do filtro, para QC visual
```

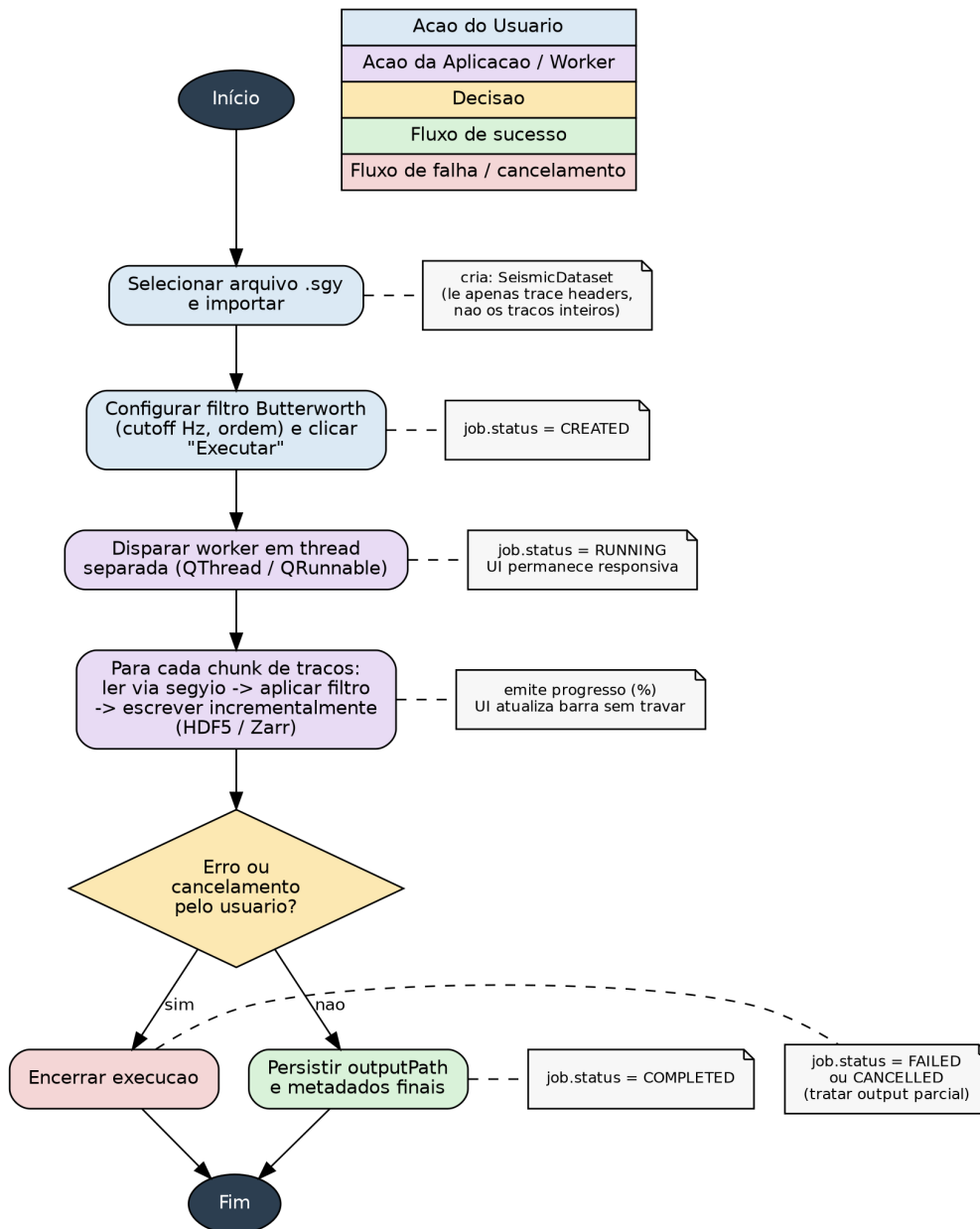
3) Escopo Core

Entidade mínima:

- **SeismicDataset**: (id, name, sourcePath, nInlines, nCrosslines, nSamples, sampleRateMs, createdAt)

A modelagem do **job de execução** (status, progresso, parâmetros do filtro) fica a critério do candidato, mas deve refletir fielmente os estados e transições descritos no diagrama abaixo. O filtro a ser implementado é um **passa-baixa Butterworth**; a UI deve expor como inputs a **frequência de corte (Hz)** e a **ordem do filtro** (inteiro, ex: 2 a 8), validando que a frequência de corte respeita a frequência de Nyquist do dado importado.

Diagrama de workflow:



Fluxo funcional da operação de filtragem, do import do SEG-Y à persistência do resultado.

Contratos mínimos da camada de negócio (desacoplada da UI, testável isoladamente com pytest):

- `create_filter_job(dataset_id, cutoff_hz, order) -> Job`
- `run_filter_job(job_id, progress_callback, cancel_token)`
- `cancel_job(job_id)`
- `get_job_status(job_id) -> Job`
- `list_jobs(dataset_id=None, status=None) -> List[Job]`

- Validações de `create_filter_job`: $0 < \text{cutoff_hz} < \text{frequência de Nyquist}$; ordem dentro de faixa razoável (ex: 2 a 8); dataset existente.
- `run_filter_job` executa a filtragem em streaming, chama `progress_callback(pct)` periodicamente e verifica `cancel_token` entre chunks (cancelamento cooperativo, não kill abrupto de thread).

Interface (PyQt5):

- Telas: Importar Sísmica, Configurar Filtro, Execução/Progresso, Histórico de Jobs.

- Fluxos: importar .sgy → configurar parâmetros → executar com barra de progresso e botão de cancelar → visualizar resultado (seção sísmica ou espectro de amplitude antes/depois) → consultar histórico com filtros por dataset/status.
- Feedback de erro/sucesso visível na UI; indicação clara do status do job.

Persistência:

- Traços: HDF5 ou Zarr, escrita incremental por chunk.
- Metadados: SQLite via SQLAlchemy (com migrations, se aplicável).

Testes:

unitários da lógica de filtragem (corretude/resposta em frequência) e da máquina de estados do job, além de 1 teste end-to-end simulando o fluxo completo import → filtro → persistência (pode usar pytest-qt ou testar a camada de negócio diretamente, sem UI real).

Requisitos técnicos explícitos (não-negociáveis):

- **UI responsiva:** a interface não pode travar/congelar durante a filtragem. Critério de aceite: deve ser possível interagir com a janela (arrastar, clicar em outro botão) durante toda a execução.
- **Aparência nativa:** a aplicação deve ter aparência nativa do PyQt5 (widgets padrão do toolkit, sem estilização customizada/CSS elaborado); o foco da avaliação é arquitetura e comportamento, não design visual.
- **Memória não-O(N) nos traços:** é proibido carregar o volume sísmico inteiro em um array (numpy ou similar) de uma vez. Leitura do .sgy e escrita do resultado devem ser feitas em streaming/chunks, com pico de memória independente do tamanho total do dado sísmico. Tenha em mente que o dado usado para validar a solução pode ser mais pesado que os datasets públicos de teste (25GB+), então a solução deve se comportar de forma estável independentemente do tamanho do arquivo de entrada.

4) Itens Opcionais (valem pontos extras)

- Paralelização da filtragem entre traços via multiprocessing (além da thread única de background);
- Visualização comparativa do espectro de amplitude antes/depois do filtro (QC);
- Cancelamento com rollback do arquivo de saída parcial;
- Log de execução persistido por job;
- Benchmark comparando memória de pico da abordagem streaming vs. uma abordagem ingênua (documentado no README).

5) Entregáveis

Repositório no **GitHub** contendo:

- Código da aplicação PyQt5;
- /tests com testes unitários da lógica de negócio (filtro, job de execução) e um teste end-to-end da interação do usuário;
- README.md com documentação do projeto, setup, comandos, decisões e trade-offs (ex: por que streaming, por que HDF5 ou Zarr).

6) Trilha de Criatividade (escolher 1)

- Limitar intervalos de inline/xline para o cálculo;
- Retomar (resume) um job interrompido a partir do último chunk processado com sucesso;
- Exportar relatório (PDF/CSV) com metadados e parâmetros do job executado;
- Descrever o motivo de escolha do item.

7) Entrega

- Prazo: até 7 dias corridos após o envio.
- Envio: link do repositório no GitHub.